BauSIM 2024 in Wien
23. September – 26. September

BIMReason - Validating BIM Model Correctness

Philipp Zech[1], Peter Burger[1], Sascha Hammes[2], David Geisler-Moroder[2] und Ruth Breu[1]

[1]*Department of Computer Science, University of Innsbruck, Tyrol, Austria*

[2]*Unit of Energy Efficient Building, University of Innsbruck, Tyrol, Austria*

Abstract

Inspections for compliance and building analysis are critical to the success of any construction endeavor. At present, such assessments are primarily conducted manually by experts in the Architecture, Engineering, and Construction (AEC) sector resulting in a tedious, labor-intensive, and generally ineffective undertaking. Yet, with the gradual adoption of Building Information Modeling (BIM), automated building analysis and compliance checking become feasible. The Industry Foundation Classes (IFC), in particular, has received a lot of traction in the AEC industry as a vendor-neutral data model. Its well-defined semantics can be exploited by reasoning engines that allow for semantic reasoning on building models, the core mechanism required for automated compliance checking and building analysis. In this paper, a general-purpose model-checking framework for IFC building models, as well as an appropriate specification layout are introduced. Model-checking via semantic reasoning is realized using various technologies from the Semantic Web. To showcase and evaluate our implementation, a sample specification is developed and tested on two IFC building models.

Introduction

The advent of digitization in the Architecture, Engineering and Construction (AEC) industry and the increased requirements for collaboration among stakeholders has led to a progressive adoption of Building Information Modeling (BIM) (Boje et al. 2020, Charef et al. 2019) over recent years. Before BIM, 2D/3D drawings, floor plans, and formal paperwork were the most common techniques in planning and implementing building projects. BIM on the other hand aims to represent a building project as a *shared digital representation of physical and functional characteristics of any built object […] which forms a reliable basis for decisions* (Kalinichuk 2015). Ideally, the resulting BIM model then comprises the 3D geometry of the building, cost information, information about energy usage, and information on more specific subjects such as Mechanical, Electrical, and Plumbing (MEP), and Heating, Ventilation, and Air Conditioning (HVAC) systems. At its core, BIM attempts to assign semantic meaning to building elements and makes this information available and interpretable by computers.

By far the most prevalent standard in the BIM environment is the Industry Foundation Classes (IFC) (Pauwels et al. 2017) which defines an open data exchange and file format for BIM, and allows for a common, vendor-neutral way to represent building data in the AEC domain. This vendor neutrality allows for the development of vendor-neutral automation tools, e.g., for the model-checking of BIM models. Currently, model-checking of BIM models that surpass clash and collision detection is generally done manually using human judgment (Amor & Dimyadi 2021). This process is inefficient and highly error-prone (Xue & Zhang 2022, Zhang & El-Gohary 2017, Lee et al. 2020). Therefore, the automation of BIM model-checking is highly relevant.

Commensurate with current issues in BIM-based model-checking, this paper introduces a general-purpose model-checking framework for BIM model analysis and compliance. Our contribution is developed as a plugin for the Eclipse framework (Eclipse Foundation 2023). The resulting framework allows IFC-based building models to be checked against various specifications. To achieve this, a checking pipeline, together with a corresponding specification language, was developed. The reasoning part was implemented using technologies from the Semantic Web, specifically through the use of Apache Jena (Apache Foundation 2023).

Our contribution is outlined as Design Science Research (DSR) (Wieringa 2014) and produces a tool as an artifact. The development of our artifact follows a systematic process, starting with gathering requirements and ending with creating prototypes and conducting experiments. The artifact is implemented as a solution to the following design science problem, outlined using the DSR template (Wieringa 2014):

Improve *BIM model quality* (context)
by designing *a vendor-neutral tool* (artifact)
that satisfies *reasoning on BIM models* (requirement)
to deliver *automated BIM model-checking*. (goal)

DSR usually refers to an *artifact* as a prototype at Technology Readiness Level 3 (TRL3), representing a conceptual solution at an early stage of technology development. Using a cyber demonstrator, our proposal achieves a TRL5 (cf. Sec. *Artifact Implementation*). The next section outlines the background of our work and state-of-the-art, followed by Sec. *Solution Proposal*, which introduces our contribution. Before we conclude in Sec. *Conclusion* we evaluate our proposal in Sec. *Results and Discussion* on two BIM models by evaluating them against an exemplary specification as developed throughout Sec. *Solution Proposal*.

Background and Related Work

Reasoning in the context of building models can be defined as the act of deducing logical conclusions from an existing building model in order to get valuable information that can be utilized for subsequent interpretation. The most common applications of reasoning in the AEC industry are compliance checking and building performance analysis.

Compliance checking is the procedure of verifying that a building model is consistent with a certain law, specification, or regulation. The normative texts for compliance checking generally specify a set of rules, that produce *true, false, unknown* or *warn* as a result (Eastman et al. 2009). Common examples of compliance checking are the assessment of fire codes, accessibility, or general codes that ensure the safety of the building. Building performance analysis on the other hand is used to evaluate how well a building operates in terms of specific criteria. This includes the analysis of energy efficiency, the computation of green building scores, evaluation of indoor air quality and acoustics (Tian et al. 2021). Building performance analysis relies on complex computations and simulations and produces a variety of different, most often numerical results. It is also not uncommon for compliance checking to enforce certain requirements on the building's performance, such as a minimum level of energy efficiency.

Compliance checking and building performance analysis is complex, and to this day, it is most often carried out manually, by domain experts. In some cases, computer software is used to assist this manual reasoning. Still, this is known to be repetitive, error-prone, inefficient, and a contributing factor to declining productivity in projects (Amor & Dimyadi 2021, Zhang & El-Gohary 2017, Gade et al. 2021). Therefore, automating this reasoning process is highly desired (Gade & Svidt 2021). With the advent of BIM and IFC, the first promising implementations for automated compliance checking and building analysis started to appear (Lee et al. 2020, Sacks et al. 2019, Sydora & Stroulia 2019). Notable stable and available implementations are the Solibri Model Checker, DesignCheck, or Fornax' ePlanCheck. Although these provide solid approaches to (semi-)automated compliance checking, they are (i) restricted in the range of specifications, i.e., checks that can be executed, (ii) implemented as a black-box, and (iii) generally do not support custom rule definition (Gade et al. 2021, Gade & Svidt 2021, Issa & Olbina 2015, Lee et al. 2020).

The specifications for compliance checking and building performance analysis are most often written in natural language. The translation of these normative texts to machine-interpretable checks is extremely complex (Amor & Dimyadi 2021). The most common reason for this is the mismatch between the information required in the specification and the information provided by the building model. As an example, many fire codes define concepts such as emergency exit routes, fire scores of building elements, and the flammability of certain materials. Therefore, mechanisms are required that help with aligning concepts from the specification to concepts in the building model (Amor & Dimyadi 2021).

To enable reasoning on BIM models, the semantics have to be understood by a reasoning engine (Hitzler & Van Harmelen 2010). Even though IFC is computer-readable, it is not directly suited for inference. To mitigate this, we employ the ifcOWL (Pauwels & Terkaj 2016), the RDF-based ontology counterpart of the IFC. The resulting representation of the BIM then can be fed as input to a reasoning engine which implement the formal inference system defined by Web Ontology Language (OWL) (Antoniou & Harmelen 2009), and by using the concepts defined in the ifcOWL ontology, additional knowledge can be derived from the underlying BIM model. However, even though reasoning on BIM models solely using the OWL semantics and ifcOWL is a powerful way to infer new information from it, supplying the reasoning procedure with additional semantics is often required (Jiang et al. 2022). For that, the Semantic Web provides the Semantic Web Rule Language (SWRL) (Horrocks et al. 2004). SWRL can be used to implement custom semantics in the form of inference rules. An inference rule consists of an *antecedent* and a *consequent* part. If in a reasoning procedure, the *antecedent* of a rule can be derived from the existing data, then the *consequent* is inferred and added for further inference. This process is repeated until no more inferences can be concluded. Rule-based reasoning is thus useful to implement recursive computations. In the context of reasoning on BIM models, it also helps with the alignment

between specification implementation and the BIM model.

In addition to rule-based reasoning, the Semantic Web also provides the Semantic Web Protocol and RDF Query Language (SPARQL) (Arenas & Pérez 2011) to extract information from RDF-based ontologies by pattern matching. However, SPARQL also specifies special `CONSTRUCT`-queries, where the results from query execution are stored back to the model on which the query was executed. This allows for query-based reasoning on RDF data. Query-based reasoning is especially useful for existential quantification, aggregation, and complex mathematical computations.

Solution Proposal

Our proposal requires two inputs: (i) an IFC-based BIM model to check, and (ii) rule specifications against which the BIM model will be evaluated. The checking engine should have the capability to check BIM models against multiple specifications. The result of the checking procedure is a list of check results. The reasoning part of the checking engine is implemented using existing technologies from the Semantic Web. The final checking engine is then packaged and deployed as a plugin for the Eclipse IDE.

Since the conditions for the desired implementations allow for different solution proposals, a requirements analysis was conducted at the beginning of the project and further reevaluated and enhanced throughout the implementation. This was crucial to narrow down the possible solution candidates. The requirements analysis was driven by our DSR problem (cf. Sec. *Introduction*), existing research in the field of automation in the AEC industry, research on the adoption of Semantic Web Technologies in the AEC industry, research on the nature and structure of normative texts, and solutions to practical problems and needs discovered during development. The following list outlines the most important requirements as determined during our analysis (in no particular order):

R1: Range of supported building models. A variety of building models supported by the verification technique is desired and a top priority. The checking engine uses BIM models as inputs, hence at least IFC models should be supported. IFC is the most advanced BIM standard and is supported by most BIM applications. The suggested verification framework only accepts single-file IFC models, although the Semantic Web can integrate multi-file and multi-standard models. As IFC data format versions (mainly IFC2x3 and IFC4) are not backward compatible, version alignment is needed. Additionally, version alignment establishes an *n:m* link between building models and specifications, enabling model verification across several specifications.

R2: Range of specifications that can be implemented. According to legal text research and building analysis research, compliance checking and building performance analysis are the most common forms of building analysis. These two analyses differ largely in calculation and findings. Building performance analysis involves computation and numerical results, while compliance verification checks specific requirements. In addition to flexible check outputs, checks should convey complicated computations and inferences. Specifications often include existential quantification, mathematical formula implementation, aggregations, and recursions. Thus, the checking engine should support complex specifications with a flexible, expressive specification layout.

R3: Simple distribution and management of specifications. Specification implementation, distribution, and maintenance should be straightforward. A specification should be a single artifact provided to the checking engine. This streamlines the import logic and improves distribution. Keeping the complete specification in one place simplifies management. Besides dissemination and maintenance, specification implementations should be transparent and verifiable by third parties.

R4: Extensible execution model. The entire verification process should be adaptable. Therefore, the checking engine implementation is simple and extendable as the BIM environment may evolve. New standards and exchange formats may make using a single IFC file as input insufficient.

R5: Performance. Time should be reasonable for checking. Performance and other needs are difficult to balance as checking huge models requires several sacrifices. First off, models need to be converted into ifcOWL. At this, caching and reducing the overall model size are not feasible as this would impede specification expressiveness and flexibility (Jiang et al. 2022). Inference is another performance-affecting step that is time-consuming on big models. In addition, support for SPARQL `CONSTRUCT`-queries is essential, as rule-based reasoning struggles with complex mathematical formulas and aggregations (Jiang et al. 2022) which we mitigate by these very SPARQL `CONSTRUCT`-queries (cf. Sec. *Specification Layout*).

Solution Overview

Our solution implements a pipeline architecture as outlined in Fig. 1. The BIM model and the specification implementation on which to check the BIM model are the two inputs to the pipeline. These are then passed through a series of steps, viz. import, inference, checking, and results extraction. The final component, viz. *Results Extractor*, produces a list of check results as output. To enable semantic reasoning, the IFC building model is translated to ifcOWL by the *Model Importer*.

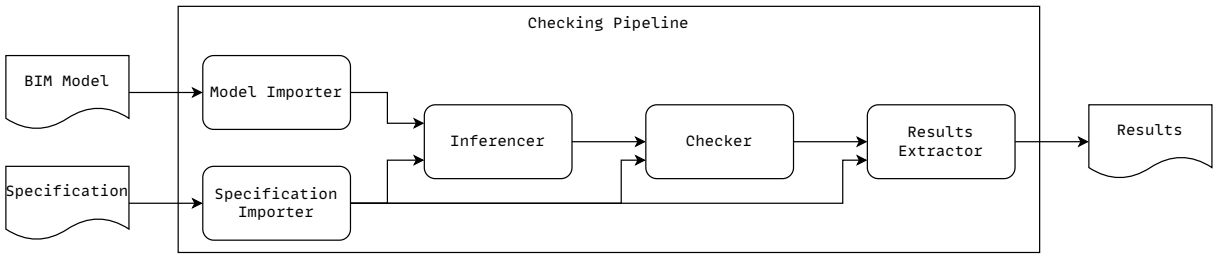

Figure 1: Pipeline of the checking procedure.

Specification Layout

As mentioned earlier, a wide range of specifications should be supported by the checking engine. To solve this, an opinionated specification layout was developed. By design, the specification implementation supplies most of the logic required for the check execution. The sole purpose of the *Checker* is then to apply this logic to the BIM model in a sensible sequence of steps. Only a few assumptions are made on how checks are implemented, and no assumptions are made on the check results. This allows for very expressive and flexible specification implementations and simplifies the checking procedure as a whole. Essentially, specification implementations are defined to be ZIP archives (.zip) compliant with the following, well-defined directory layout (cf. Fig. 4):

checks/{check}.jr The `checks/` directory contains a list of check implementations in the form of Jena Rules (.jr). There can be arbitrarily many rule files in this directory. These rules are used to implement the concrete checks (rules from normative texts), which generate the final check results. The naming of the files is open to the implementer of the specification, yet it is advised to name them according to the actual checks they implement. There can be multiple check implementations per file. The checking rules themselves are definite Horn clauses in implication form (Giordano et al. 1992) and composed of an antecedent and a consequent. The antecedent is used to define the objects to be tested, whereas the consequent defines the check result that is applied to the object. Fig. 2 shows an example of such a check implementation. In this specific case, the antecedent of the rule defines a pattern that matches every element `?s` of type `myspec:door`, where `?s` has a property `myspec:hasWidth` with a corresponding value `?w` that is less than `"813"`. The task of a rule-based reasoning engine is then to find all possible assignments of `?s` and `?w` that satisfy this pattern. For each solution, the consequent of the rule is evaluated and added to the model. In this case, the consequent assigns `myspec:FAIL` to the tested door (`?s`) on the check `myspec:check_IBC_2015_Section_1010_b`. Conceptually, this rule describes, that every door that is less than 813mm wide fails the check defined in the IBC 2015, Section 1010b. For more details, see Sec. *Checker*.

```
[someCheck:
    (?s rdf:type myspec:Door)
    (?s myspec:hasWidth ?w)
    lessThan(?w, "813"^^xsd:double)
        -> (?s myspec:check_IBC_2015_Section_1010_b
     myspec:FAIL)]
```

Figure 2: An excerpt of an exemplary .jr file used for check implementation.

inference/ontologies/{graph}.ttl The `ontologies/` sub-directory within the `inference/` directory contains an arbitrary number of RDF graphs stored as Turtle (.ttl) files. There can be arbitrary many of these files, and the naming of them is open to the specification implementer. Also, there is no restriction to the content of the Turtle files. They are imported during the inference step and appended to the building model. This could be used to import external schemata and ontologies for providing additional semantics on top of the ifcOWL. For more details, see Sec. *Inferencer*.

inference/rules/{rule}.jr The `inference/rules/` directory contains an arbitrary number of rule files (.jr), which in turn contain the rules used for rule inference. Similar to the rules in the `checks/` directory, the naming of them is open to the implementer. These rules are used to enrich the building model with additional information, including the implementation of the OWL semantics and IFC version alignment. A concrete example of IFC version alignment is shown in Fig. 3. Here, two rules are used to map different *IfcDoor* implementations (IFC2x3 and IFC4) to a common `myspec:Door`. For more details, see Sec. *Inferencer*.

```
[(?x rdf:type ifc2_3:IfcDoor) -> (?x rdf:type myspec:
    Door)]
[(?x rdf:type ifc4:IfcDoor) -> (?x rdf:type myspec:
    Door)]
```

Figure 3: Implementation of IFC version alignment using rule files from the `inference/rules/` directory.

inference/queries/{query}.sparql The `inference/queries/` directory contains an arbitrary number of SPARQL-queries used for query inference. The naming of the queries is open to the implementer. The content of the SPARQL files has to be a valid SPARQL `CONSTRUCT`-query. Other than that, there are no restrictions on the concrete implementation of the queries. The queries in this directory are used to implement inferences that cannot be realized using rule-based inference, such as complex, mathematical computations, existential quantification, and aggregations. For more details, see Sec. *Inferencer*.

extraction/extraction.sparql The `extraction/` directory contains a mandatory `extraction.sparql` file. Once check execution is completed, this query is used to extract the final check results from the model. Other than being a valid SPARQL `SELECT`-query, the exact implementation is open to the implementer of the specification. However, it is common to implement the extraction query in such a way, that each element of the result list contains information about the object that was tested, an additional identified for the object, the check on which the object was tested, and the final check result. The results from the extraction query form the output of the checking procedure. For more detail, see Sec. *Results Extractor*.

Fig. 4 shows the layout of a possible specification implementation. For an actual specification implementation, see Sec. *Results and Discussion*.

Artifact Implementation

Many software libraries provide support for technologies from the Semantic Web. In our implementation, Apache Jena was used, as it provides a general platform for many Semantic Web technologies including reading, writing, and manipulating RDF data, support for rule-based reasoning, a custom rule language similar to SWRL, an implementation of the OWL semantics through rule-based reasoning, and query-based reasoning using SPARQL `CONSTRUCT`-queries.

```
specification.zip
    checks/
        check1.jr
        check2.jr
    inference/
        ontologies/
            rdf1.ttl
            rdf2.ttl
        rules/
            rules1.jr
            rules2.jr
            rules3.jr
        queries/
            query1.sparql
            query2.sparql
    extraction/
        extraction.sparql
```

Figure 4: Directory structure and corresponding files of a specification implementation.

Specification Importer

Importing a specification, together with model import, forms the first stage in our pipeline. Theoretically, specification import and model import could be executed in parallel. However, importing the specification first results in better *fail fast* performance in case of an erroneous specification. Specifically, the *Specification Importer* reads and parses the specification thereby parsing the supplied rules, queries, and RDF graphs to their respective objects in Apache Jena. If the supplied specification file does not conform to the layout defined in Fig. 4, it is rejected. The final result of specification import is an in-memory representation of the supplied specification as required in later steps.

Model Importer

After specification import is completed, the IFC-based BIM model is imported. As no assumptions are made on the BIM model, any valid IFC file is considered a valid input. The IFC-based BIM model is then converted into ifcOWL and forwarded to Apache Jena for in-memory storage. The final result of this step is a single RDF graph, containing the BIM model as used in later steps of the pipeline.

Inferencer

Once specification and model are imported, semantic enrichment is performed on the BIM model by using the inference section (`inference/`) from the imported specification. The idea behind the inference step is to enrich the BIM model with additional information, that can then be used in the check execution. Generally, inference is divided into three steps, each dedicated to a different type of inference, viz. additional ontology import, rule inference, and query inference. These steps are ex-

ecuted in sequence, whereby the order is crucial as the inference results of previous steps can be used in subsequent steps. In the proposed implementation, execution is ordered by *generality*: (i) additional ontology import, (ii) rule inference, and (iii) query inference, whereby ontology import is the most general, and query inference is the most specific.

Checker

Checks are implemented as Jena rules and provided in the imported specification (`checks/{check}.jr`). The antecedent of a checking rule defines the pattern of objects that are subject to a specific check. The consequent is then implemented in such a way, that the inference thereof represents the result of applying the check on the object. This is achieved by defining the consequent of a checking rule as a `(?object, ?check, ?result)` triple. The `?object` represents the object that is checked, `?check` is an identifier for the concrete check that is applied to the object, and the check result is captured by `?result`. Using the `GenericRuleReasoner` from Jena, these rules are executed on the enriched model (cf. Sec. *Inferencer*). The inferred statements are then considered the check results. Although checks could also be implemented directly in the inference step, outsourcing check execution into a different step ensures that all the model enrichment is performed before the checking rules are applied.

Results Extractor

The *Result Extractor* extracts the final results after check execution using the SPARQL extraction query provided in the specification (`extraction/extraction.sparql`). The exact implementation of how and what results are extracted is open to the implementer of the specification. For example, for our evaluation in Sec. *Results and Discussion*, each check was specifically declared to be of type `spec:Check`. The extraction query then simply selects all the triples from the model, wherein the `?predicate` in `(x, ?predicate, y)` is of type `spec:Check`. It is also worth mentioning, that the final check results no longer have to be RDF triples. For example, it might be useful to append the IFC-ID of tested objects to their respective check results, making it a query solution with 4 fields (the RDF identifier of the tested object, the IFC identifier of the tested object, the RDF identifier of the check, and the check result). As check result extraction is the last step in the pipeline, its results are considered to be the output of the entire checking procedure.

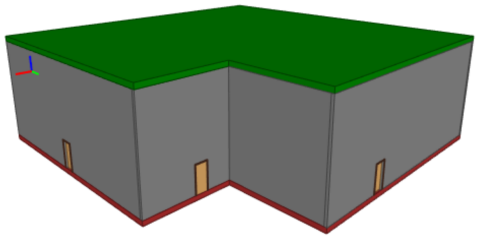

Figure 5: The simple model used for testing.

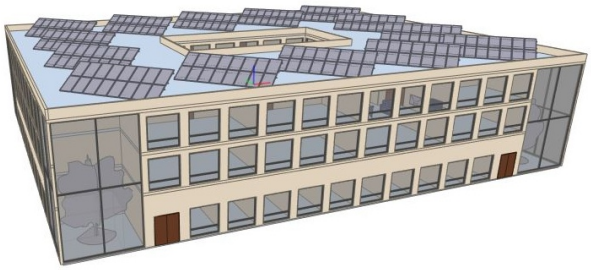

Figure 6: The complex model used for testing.

Results and Discussion

To evaluate our proposal, a sample specification was developed and tested on two different IFC building models. The models were obtained from an open repository[1]. The first building model shown in Fig. 5 is a simple, single-story building that lacks any interior walls. The IFC version of this model is IFC4. The second building model shown in Fig. 6 is a large, multi-story office building. The IFC version used for this building model is IFC2x3. Different IFC versions were chosen to enforce version alignment within the specification implementation. The used models, specifications and implementation are available for download at https://www.github.com/phizech/modelchecker. In the following, we report on the performance of our proposal. All experiments were conducted on a Windows 10 x64 machine running on an Intel(R) Core(TM) i7-4790 CPU@3.60GHz with 16GB DDR3 RAM. It is expected, that the time for checking increases with model size.

Tbl. 1 shows the detailed performance evaluation of the checking procedure on both the small and the large building model. In summary, the checking procedure on the smaller building model took a total of 5.035 secs, whereas the checking on the larger building model took a total of 124.018 secs. In both cases, the model enrichment step takes the longest, with 3.432 secs for the smaller and 87.002 secs for the larger model. Considering the actual size of the models (small: 0.269MB, large: 13.625MB), the correlation between model

[1] http://smartlab1.elis.ugent.be:8889/IFC-repo/

size and time taken for enrichment appears to be super-linear. This is also reflected in the size of the RDF graph before and after enrichment. For the small model, the size of the RDF graph approximately doubled from 37831 statements to 73619 statements after enrichment, whereas for the large model, the RDF graph increased by a factor of four, from 1796200 statements to 7715966 statements.

The time for specification import is independent of the model used, with 0.593 secs for the smaller model, and 0.735 secs for the large model. The time required for model import is expected to be linearly proportional to the model size. However, since this step downloads the ifcOwl ontology and several other transitive dependencies from the internet, the import of small building models could also require several secs. In both cases, the model import was the most expensive step after enrichment and took 3.432 secs for the small model and 30.975 secs for the large model.

Check execution took 0.017 secs for the small model and 1.401 secs for the large model. The time taken for result extraction is almost negligible, with 0.010 secs for the small model and 0.034 secs for the large model. The checking process resulted in a total of 23 final check results for the small model and 637 for the large model. Fig. 7 shows a screenshot of the current report layout. Admittedly, the result presentation merits improvement, yet it already provides crucial information regarding which BIM model elements do not fulfill the earlier specified check (cf. Fig. 2).

Table 1: Performance measurements.

	Simple model	Complex model
IFC version	IFC4	IFC2x3
model size (MB)	0.269	13.625
specification import (secs)	0.593	0.735
model import (secs)	3.432	30.975
enrichment duration (secs)	0.934	87.002
checking (s)	0.017	1.401
result extraction (s)	0.010	0.034
logging and misc (s)	0.049	3.871
total duration (s)	5.035	124.018
RDF triples after import	37831	1796200
RDF triples after enrichment	73619	7715966
RDF triples after checking	73642	7716336
number of check results	23	637

In synopsis, our proposal delivers advances to existing work by introducing a generic, open-source platform for model checking of BIM models that (i) allows for the definition of custom rules, (ii) provides full transparency regarding which rules are implemented how, and (iii), supports checking BIM models of considerable size (cf. Fig. 6) in a reasonable time. Currently, the specification needs to be developed manually, yet, as suggested by Zhang & El-Gohary (2017) and Zhou & El-Gohary (2021), Natural Language Processing (NLP) provides an avenue for the automated generation of specifications from regulation bodies.

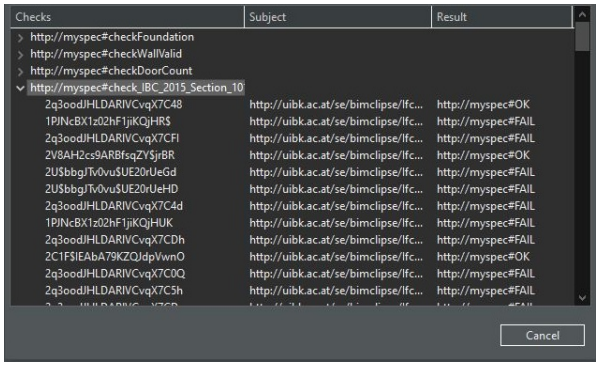

Figure 7: Results of evaluating the sample check from Fig. 2 against the BIM model from Fig. 6.

Conclusion

In this paper, a general-purpose checking framework for the analysis of IFC-based BIM models was introduced. Alongside the checking engine, an opinionated layout for specification implementations was developed. The checking engine receives an IFC building model and a specification implementation as input. The specification is then tested on the imported building model using semantic reasoning. The output of the checking procedure is a list of check results. To showcase the functionality, a sample specification was developed and tested on two BIM models with different IFC versions. The evaluation of the checking procedure on both building models shows, that the proposed implementation can successfully test specifications on different building models in a reasonable amount of time. In future work we plan to improve our solution's performance and reporting facilities and investigate the application of NLP for the automated generation of specifications from building standards.

Acknowledgment

The research leading to these results has received funding from the Austrian Research Promotion Agency (FFG) under Grant Agreement No.: 898708, TwinLight.

References

Amor, R. & Dimyadi, J. (2021), 'The promise of automated compliance checking', *Developments in the Built Environment* **5**, 100039.

Antoniou, G. & Harmelen, F. v. (2009), 'Web ontology language: OWL', *Handbook on ontologies* pp. 91–110.

Apache Foundation (2023), 'Apache Jena'. **URL:** *https://jena.apache.org/*

Arenas, M. & Pérez, J. (2011), Querying semantic web data with SPARQL, *in* 'Proceedings of the thirtieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems', pp. 305–316.

Boje, C., Guerriero, A., Kubicki, S. & Rezgui, Y. (2020), 'Towards a semantic Construction Digital Twin: Directions for future research', *Automation in Construction* **114**, 103179.

Charef, R., Emmitt, S., Alaka, H. & Fouchal, F. (2019), 'Building Information Modelling adoption in the European Union: An overview', *Journal of Building Engineering* **25**, 100777.

Eastman, C., min Lee, J., suk Jeong, Y. & kook Lee, J. (2009), 'Automatic rule-based checking of building designs', *Automation in Construction* **18**(8), 1011–1033.

Eclipse Foundation (2023), 'Eclipse IDE'. **URL:** *https://www.eclipse.org/ide/*

Gade, P. N., Jensen, R. L. & Svidt, K. (2021), Practitioner experiences and requirements for rule translation used for building information model-based model checking, *in* 'Cooperative Design, Visualization, and Engineering: 18th International Conference, CDVE 2021, Virtual Event, October 24–27, 2021, Proceedings 18', pp. 84–96.

Gade, P. N. & Svidt, K. (2021), 'Exploration of practitioner experiences of flexibility and transparency to improve BIM-based model checking systems', *Journal of Information Technology in Construction* **26**, 1041–1060.

Giordano, L., Martelli, A. & Rossi, G. (1992), 'Extending Horn clause logic with implication goals', *Theoretical Computer Science* **95**(1), 43–74.

Hitzler, P. & Van Harmelen, F. (2010), 'A reasonable semantic web', *Semantic Web* **1**(1-2), 39–44.

Horrocks, I., Patel-Schneider, P. F., Boley, H., Tabet, S., Grosof, B., Dean, M. et al. (2004), 'SWRL: A semantic web rule language combining OWL and RuleML', *W3C Member submission* **21**(79), 1–31.

Issa, R. R. A. & Olbina, S., eds (2015), *Building Information Modeling: Applications and Practices*.

Jiang, L., Shi, J. & Wang, C. (2022), 'Multi-ontology fusion and rule development to facilitate automated code compliance checking using BIM and rule-based reasoning', *Advanced Engineering Informatics* **51**, 101449.

Kalinichuk, S. (2015), 'Building information modeling comprehensive overview', *Journal of Systems Integration* **6**(3), 25.

Lee, Y.-C., Ghannad, P., Dimyadi, J., Lee, J.-K., Solihin, W. & Zhang, J. (2020), A comparative analysis of five rule-based model checking platforms, *in* 'Construction Research Congress 2020', pp. 1127–1136.

Pauwels, P. & Terkaj, W. (2016), 'EXPRESS to OWL for construction industry: Towards a recommendable and usable ifcOWL ontology', *Automation in construction* **63**, 100–133.

Pauwels, P., Zhang, S. & Lee, Y.-C. (2017), 'Semantic web technologies in AEC industry: A literature overview', *Automation in Construction* **73**, 145–165.

Sacks, R., Bloch, T., Katz, M. & Yosef, R. (2019), Automating design review with artificial intelligence and BIM: State of the art and research framework, *in* 'ASCE International Conference on Computing in Civil Engineering 2019', pp. 353–360.

Sydora, C. & Stroulia, E. (2019), Towards rule-based model checking of building information models, *in* 'ISARC. Proceedings of the International Symposium on Automation and Robotics in Construction', Vol. 36, pp. 1327–1333.

Tian, Z., Zhang, X., Wei, S., Du, S. & Shi, X. (2021), 'A review of data-driven building performance analysis and design on big on-site building performance data', *Journal of Building Engineering* **41**, 102706.

Wieringa, R. (2014), *Design Science Methodology for Information Systems and Software Engineering*, Springer Berlin Heidelberg.

Xue, X. & Zhang, J. (2022), 'Regulatory information transformation ruleset expansion to support automated building code compliance checking', *Automation in Construction* **138**, 104230.

Zhang, J. & El-Gohary, N. M. (2017), 'Integrating semantic NLP and logic reasoning into a unified system for fully-automated code checking', *Automation in Construction* **73**, 45–57.

Zhou, P. & El-Gohary, N. (2021), 'Semantic information alignment of BIMs to computer-interpretable regulations using ontologies and deep learning', *Advanced Engineering Informatics* **48**, 101239.